C++ PROGRAMMING

100 Programs from Beginner to Advanced

CHAPTER 1

C++ Basics and Input/Output

10 Programs • Beginner Level

Standard: C++17
Compiler: GCC / G++
Editor: Visual Studio Code

Companion documentation for the C++ Programming Portfolio

Table of Contents

Chapter Introduction 3
Program 01 - Hello World 5
Program 02 - Multi-line Output and Escape Sequences 8
Program 03 - Variable Declaration and Initialization 11
Program 04 - Sum of Two Integers 14
Program 05 - Product of Two Floating-Point Numbers 16
Program 06 - Swap Two Numbers Using a Third Variable 18
Program 07 - Area and Perimeter of a Rectangle 21
Program 08 - Celsius to Fahrenheit Converter 23
Program 09 - Size of Primitive Data Types 26
Program 10 - ASCII Value of a Character 29
Chapter Summary 32
Practice Exercises - Round-Up 33
Conclusion and Next Steps 34

Chapter 1 - C++ Basics and Input/Output

Welcome to Chapter 1 of the C++ Programming - 100 Programs course. This chapter is your first step into the world of C++ and is designed for absolute beginners. Over the next ten programs you will learn the basic structure of every C++ program, how to send text to the screen, how to read data typed by the user, how variables of different types are declared and used, and how characters are represented inside the computer.

Each program in this chapter is a self-contained lesson. It begins with an aim and a problem statement, walks through an algorithm in plain English, presents the full source code with rich comments, and then explains every significant line in beginner-friendly depth. You will then see the actual input and output captured by running the program, read about where the same pattern is used in real software, learn about the mistakes beginners typically make, and finally tackle a small practice question that extends the program.

The chapter uses only the <iostream> header and the standard namespace std. You will meet the four foundational tools of C++ console programming: **cout** (character output, sent to the screen), **cin** (character input, read from the keyboard), the **stream insertion operator <<** (push data into cout), and the **stream extraction operator >>** (pull data out of cin). You will also learn about variables, primitive data types (int, float, double, char, bool), the sizeof operator, escape sequences, comments and the char-to-int conversion that underlies all text processing.

By the end of this chapter you will be able to write, compile and run a complete C++ program that interacts with the user, performs simple arithmetic, and prints well-formatted results. These skills are the foundation on which every later chapter - operators, loops, functions, arrays, pointers, classes and the STL - is built.

Chapter Learning Outcomes

1. Understand the structure of a C++ program - preprocessor directive, namespace, main function, statements.
2. Use cout and the << operator to print text, numbers and mixed output.
3. Use cin and the >> operator to read integers, doubles and characters from the keyboard.
4. Declare and initialize variables of all primitive types: int, float, double, char, bool.
5. Use escape sequences (\n, \t, \\, \") to format output across multiple lines.
6. Avoid the integer-division trap when writing arithmetic formulas.
7. Use the sizeof operator to inspect type sizes on your platform.
8. Convert between char and int to print ASCII values and perform simple character arithmetic.

Program 01 - Hello World

Program Number	1
Title	Hello World
Difficulty	Beginner
File	01_*.cpp

1. Aim

To write, compile and execute the first C++ program that prints a greeting message on the screen, and to understand the basic structure of every C++ program.

2. Concepts Used

1. #include preprocessor directive
2. <iostream> header
3. std namespace
4. main() function - the program entry point
5. cout (standard output stream)
6. << (stream insertion operator)
7. endl (newline + buffer flush)
8. return statement

3. Problem Statement

Write a C++ program that displays the message "Hello, World!" on the standard output (console).

4. Algorithm

1. Include the input-output stream header <iostream>.
2. Bring the standard namespace std into scope to avoid writing std:: each time.
3. Define the main() function - the program entry point.
4. Inside main(), use cout to print "Hello, World!".
5. End the line with endl and return 0 to indicate success.

5. Source Code

```
#include <iostream> using namespace std; int main() { cout << "Hello, World!" << endl; return 0; }
```

6. Line-by-Line Explanation

#include <iostream>

This is a **preprocessor directive**. The # symbol tells the compiler's preprocessor to run this line *before* actual compilation. *#include <filename>* literally copies the contents of the named header file into your source code. <iostream> is the standard C++ header that declares the input/output stream objects cin (console input), cout (console output), cerr (error output) and endl (newline + flush). Without this line, the compiler would not know what cout is.

using namespace std;

C++ groups names (functions, classes, variables) into containers called **namespaces** to prevent name clashes in large projects. Everything in the standard library lives inside the namespace std, so the full name of cout is actually std::cout. Writing *using namespace std;* imports the entire std namespace into the current file so we can simply write cout. The semicolon ends the statement - every C++ statement must end with a semicolon.

int main()

This declares the **main function**. Every C++ program must have exactly one main() - the operating system calls it when the program starts. int before main says this function returns an integer to the OS. The empty parentheses () mean the function takes no arguments.

{ ... }

These curly braces mark the **body** of main(). Everything between them is what the function executes.

cout << "Hello, World!" << endl;

cout is the standard output stream object representing the console. The << operator is the **stream insertion operator** - it sends the value on its right into the stream on its left, and can be chained. "Hello, World!" is a **string literal** enclosed in double quotes. endl is a **manipulator** that inserts a newline character and flushes the output buffer so the text appears immediately.

return 0;

Returns the integer 0 to the operating system. By convention, 0 means "the program finished successfully"; any non-zero value indicates an error. Modern C++ allows you to omit return 0; in main() only - the compiler inserts it implicitly - but writing it explicitly is good practice.

7. Sample Input / Output

Input	(none)
Output	`Hello, World!`

8. Real-World Application

Where this is used

Every C++ project - be it an operating system kernel module, a game engine, a desktop application, or firmware for an embedded device - begins from this minimal structure of #include, main(), and I/O. The Hello World program is the canonical smoke test used worldwide to verify that the compiler toolchain (preprocessor, compiler, linker, runtime) is correctly installed and working.

9. Common Errors

Watch out for

- Forgetting the semicolon ; at the end of a statement.
- Typing cout as Cout or COUNT (C++ is case-sensitive).
- Writing #include <iostream.h> (old Turbo C++ style; not valid in C++17).
- Omitting the namespace std (then cout must be written as std::cout).
- Using single quotes 'Hello' instead of double quotes "Hello".

10. Practice Question

Try this

Modify the program to print your **full name** and your **college name** on two separate lines using a single cout statement. Hint: chain endl between two strings: cout << "My Name" << endl << "My College" << endl;

Program 02 - Multi-line Output and Escape Sequences

Program Number	2
Title	Multi-line Output and Escape Sequences
Difficulty	Beginner
File	02_*.cpp

1. Aim

To learn how to print text on multiple lines, insert tabs and quotes inside output, and write comments - using C++ escape sequences.

2. Concepts Used

1. cout with chained << operators
2. Escape sequences: \n, \t, \\, \", \r
3. Single-line comments //
4. Multi-line comments (slash-star ... star-slash)
5. Difference between endl and \n

3. Problem Statement

Write a C++ program that prints a short student profile containing the user's name, course, a tabular subject list, a quoted sentence and a Windows-style file path - using escape sequences to format the output across multiple lines.

4. Algorithm

1. Include <iostream> and bring in the std namespace.
2. Inside main(), print a heading using \n for line breaks.
3. Print rows of columns separated by \t (tab).
4. Print a quoted sentence using \" to embed double quotes.
5. Print a Windows file path using \\ for each backslash.
6. return 0.

5. Source Code

```
#include <iostream> using namespace std; int main() { cout << "Student Profile\n";
cout << "Name: Aman Verma\n"; cout << "Course: B.Tech (CSE)\n\n"; cout <<
"Subject\t\tMarks\n"; cout << "Maths\t\t95\n"; cout << "Physics\t\t88\n\n"; cout <<
"Quote: \"C++ is fun!\"\n"; cout << "File path: C:\\CPP\\hello.cpp\n"; return 0; }
```

6. Line-by-Line Explanation

cout << "Student Profile\n";

The string contains the escape sequence \n. When cout sees \n it moves the cursor to the beginning of the next line instead of printing a backslash and an n. This is how we print on multiple lines without writing many cout statements.

cout << "Subject\t\tMarks\n";

\t is the **horizontal tab** escape sequence. It moves the cursor to the next tab stop (usually every 8th column). Two tabs \t\t push "Marks" far enough right to form a neat column with the data rows below it.

cout << "Quote: \"C++ is fun!\"\n";

A double quote " normally ends a string literal. To *print* a double quote we must escape it with a backslash: \". The output therefore reads exactly Quote: "C++ is fun!".

cout << "File path: C:\\CPP\\hello.cpp\n";

In Windows, file paths use backslashes. But \ starts an escape sequence in C++ string literals - so \C could be misread (and \n inside a path would print as a newline!). To print a literal backslash, we write \\ (escape the backslash itself). The output is File path: C:\CPP\hello.cpp.

// single-line comment

Anything after // on a line is ignored by the compiler. Use it for short notes.

endl vs \n

endl and \n both move to the next line, but endl also **flushes** the output buffer (forces the OS to write pending output immediately). In performance-sensitive code, prefer \n - flushing on every line is slow.

7. Sample Input / Output

Input	(none)
Output	`Student Profile Name: Aman Verma Course: B.Tech (CSE) Subject Marks Maths 95 Physics 88 Quote: "C++ is fun!" File path: C:\CPP\hello.cpp`

8. Real-World Application

Where this is used

Formatted console output is the foundation of **command-line tools**, **log file writers**, **terminal dashboards** and **report generators**. Escape sequences are used everywhere - installers print colored progress bars (via \033[...m), database dumps align columns with \t, and Windows service loggers double every backslash in file paths. The same skills apply when generating HTML/XML or building text-based menus in embedded systems that lack a GUI.

9. Common Errors

Watch out for

- Writing C:\new in a string - prints as C:ew because \n is interpreted.
- Forgetting to escape \\ inside Windows file paths.
- Mixing endl and \n inconsistently.
- Long cout chain missing trailing semicolon.
- Using "C++ is fun!" inside a string without escaping the inner quotes.

10. Practice Question

Try this

Write a program that prints the ASCII art of a triangle using escape sequences and a single cout statement: /\ / \ /____\ Hint: you need to escape each backslash. Use leading spaces for alignment and \n between lines.

Program 03 - Variable Declaration and Initialization

Program Number	3
Title	Variable Declaration and Initialization
Difficulty	Beginner
File	03_*.cpp

1. Aim

To understand the primitive data types in C++ and to declare, initialize and print variables of different types using the sizeof operator.

2. Concepts Used

1. Primitive types: int, float, double, char, bool
2. Variable declaration and initialization
3. sizeof operator (compile-time)
4. f suffix for float literals
5. cout printing different types

3. Problem Statement

Declare one variable each of type int, float, double, char and bool. Initialize them with sample values and print each variable along with its size in bytes using the sizeof operator.

4. Algorithm

1. Include <iostream> and bring in the std namespace.
2. Declare and initialize variables of all five primitive types.
3. For each variable, print its name, value and size using cout and sizeof.
4. return 0.

5. Source Code

```
#include <iostream> using namespace std; int main() { int age = 42; float pi = 3.14f; double euler = 2.71828; char grade = 'A'; bool isPass = true; cout << "Integer : value = " << age << ", size = " << sizeof(age) << " bytes\n"; cout << "Float : value = " << pi << ", size = " << sizeof(pi) << " bytes\n"; cout << "Double : value = " << euler << ", size = " << sizeof(euler) << " bytes\n"; cout << "Character: value = " << grade << ", size = " << sizeof(grade) << " byte\n"; cout << "Boolean : value = " << isPass << ", size = " << sizeof(isPass) << " byte\n"; return 0; }
```

6. Line-by-Line Explanation

int age = 42;

Declares a variable named age of type int (a 32-bit signed integer) and initializes it with 42. The compiler allocates 4 bytes of memory and stores the binary representation of 42. int typically holds values from about -2.1 billion to +2.1 billion.

float pi = 3.14f;

A float is a **single-precision** floating-point number occupying 4 bytes, giving roughly 7 significant decimal digits. The f suffix on 3.14f is important: without it, 3.14 is treated as a double, and assigning a double to a float may cause a compiler warning about loss of precision.

double euler = 2.71828;

A double is a **double-precision** floating-point number occupying 8 bytes, giving roughly 15 significant digits. It is the default choice when you need decimals and is preferred over float for most scientific and financial calculations.

char grade = 'A';

A char stores a single character in **single quotes**. Internally it stores the ASCII code for A, which is 65. Characters and small integers are interchangeable in C++. Note: 'A' (single quotes) is a char; "A" (double quotes) is a string literal - the two are different types!

bool isPass = true;

A bool stores one of two values: true or false. When printed with cout, true appears as 1 and false appears as 0. Use cout << boolalpha to print the words instead.

sizeof(age)

sizeof is a **compile-time operator** (not a function) that returns the size in bytes of its operand. The result is a constant of type size_t. Using sizeof makes your code portable - sizes can differ between platforms.

7. Sample Input / Output

Input	(none)
Output	`Integer : value = 42, size = 4 bytes Float : value = 3.14, size = 4 bytes Double : value = 2.71828, size = 8 bytes Character: value = A, size = 1 byte Boolean : value = 1, size = 1 byte`

8. Real-World Application

Where this is used

Choosing the right data type is a daily engineering decision: **sensor data** in IoT firmware often fits in a float to save RAM; **currency calculations** must use double to avoid rounding errors; **status flags** should be bool for clarity; **character codes** underlie all text processing; **array indices** use int or size_t. Knowing the size of each type matters in embedded systems, network protocols and binary file formats where every byte is laid out explicitly.

9. Common Errors

Watch out for

• Wrote char c = "A"; (double quotes) instead of char c = 'A';
• Assigned 3.14 (a double literal) to a float without the f suffix - causes precision warning.
• Wrote cout << 'Hello'; (single quotes around multi-char) - not a string.
• bool prints as 0/1 instead of true/false - use cout << boolalpha.
• Assumed int is always 4 bytes - on some 16-bit MCUs it is 2 bytes.

10. Practice Question

Try this

Add a variable of type unsigned int and one of type long long. Print their sizes and try to store a value larger than 2 billion in each (e.g. 3000000000). Observe what happens if you store a value larger than INT_MAX into a plain int.

Program 04 - Sum of Two Integers

Program Number	4
Title	Sum of Two Integers
Difficulty	Beginner
File	04_*.cpp

1. Aim

To take two integers as input from the user and display their sum, introducing the cin input stream and the + arithmetic operator.

2. Concepts Used

1. Variable declaration and initialization
2. cin (standard input stream)
3. >> (stream extraction operator)
4. Arithmetic operator +
5. Prompting the user before reading input

3. Problem Statement

Write a C++ program that reads two integers from the keyboard and prints their sum.

4. Algorithm

1. Declare two int variables a and b, plus a third variable sum.
2. Prompt the user with cout.
3. Read the two integers using cin >> a >> b.
4. Compute sum = a + b.
5. Print the result using cout.

5. Source Code

```
#include <iostream> using namespace std; int main() { int a = 0, b = 0, sum = 0; cout << "Enter two integers: "; cin >> a >> b; sum = a + b; cout << "Sum = " << sum << endl; return 0; }
```

6. Line-by-Line Explanation

int a = 0, b = 0, sum = 0;

Three int variables are declared and initialized to 0 in a single statement, separated by commas. **Initializing variables at declaration is best practice** - it avoids the classic bug of reading from an uninitialized variable, which produces undefined behavior (random garbage values).

cout << "Enter two integers: ";

A **prompt** is printed so the user knows the program is waiting for input. Without a prompt, the program appears frozen and the user does not know what to type.

cin >> a >> b;

cin (pronounced "see-in") is the standard input stream object, attached to the keyboard. >> is the **stream extraction operator** - it pulls data out of the stream and stores it into the variable on its right. The two extractions can be chained: the first integer goes into a, the second into b. cin skips leading whitespace automatically and stops reading a number when it hits a non-digit character.

sum = a + b;

The + operator adds the two values. The result is stored into sum via the assignment operator =. The right-hand side is evaluated first (a + b), then assigned to the left-hand side.

cout << "Sum = " << sum << endl;

Three things are pushed into cout in sequence: the literal string "Sum = ", the value of sum, and a newline. The output looks like Sum = 19.

7. Sample Input / Output

Input	12 7
Output	`Enter two integers: Sum = 19`

8. Real-World Application

Where this is used

Reading input from a user and producing a computed result is the **template for every interactive program**: calculators read two numbers and an operator; billing systems read quantities and prices; statistical aggregators read data points and sum / average them; POS terminals read item codes and accumulate totals. The same cin >> ... >> ... pattern scales up to file and network streams in later chapters.

9. Common Errors

Watch out for

• No prompt before cin - the program appears frozen.
• Wrote cin << a (wrong direction) - use >> with cin.
• Variable uninitialized - initialize at declaration: int a = 0;
• Reads only one number when user presses Enter between numbers - call cin twice or read both on one line.
• Crash on non-integer input - cin enters fail state (handled in Chapter 9).

10. Practice Question

Try this

Extend the program to read **three** integers and print both their **sum** and **average** (the average should be a double). Hint: use sum / 3.0, not sum / 3, to avoid integer division!

Program 05 - Product of Two Floating-Point Numbers

Program Number	5
Title	Product of Two Floating-Point Numbers
Difficulty	Beginner
File	05_*.cpp

1. Aim

To take two floating-point numbers from the user and print their product, demonstrating the * operator and double arithmetic.

2. Concepts Used

1. double data type (preferred over float)
2. cin >> reading floating-point values
3. Arithmetic operator *
4. Integer vs floating-point division
5. Default precision of cout for double

3. Problem Statement

Write a C++ program that reads two floating-point numbers from the keyboard and displays their product.

4. Algorithm

1. Declare two double variables a, b, and a product variable.
2. Prompt the user.
3. Read the two doubles with cin >> a >> b.
4. Compute product = a * b.
5. Print the product.

5. Source Code

```
#include <iostream> using namespace std; int main() { double a = 0.0, b = 0.0, product = 0.0; cout << "Enter two floating-point numbers: "; cin >> a >> b; product = a * b; cout << "Product = " << product << endl; return 0; }
```

6. Line-by-Line Explanation

double a = 0.0, b = 0.0, product = 0.0;

Three double variables are declared and initialized. double is a 64-bit floating-point type giving about 15 significant decimal digits - enough for almost all general-purpose computation. We initialize with 0.0 (a double literal) rather than 0 (an int literal) to make intent clear, although the compiler would convert automatically.

cin >> a >> b;

cin reads floating-point numbers much like integers - it skips whitespace, then reads digits, an optional decimal point, an optional e exponent (e.g. 1.5e3 for 1500), and stops at the first non-numeric character.

product = a * b;

The * operator performs floating-point multiplication because both operands are double. If one operand were int, C++ would implicitly convert it to double first ("usual arithmetic conversions") - but if *both* were int, the result would be int and any fractional part would be discarded.

cout << "Product = " << product << endl;

By default, cout prints doubles with up to 6 significant digits and trims trailing zeros. To control precision you would include <iomanip> and write cout << fixed << setprecision(4);. For now, the default is fine.

7. Sample Input / Output

Input	2.5 4.0
Output	`Enter two floating-point numbers: Product = 10`

8. Real-World Application

Where this is used

Floating-point multiplication is everywhere: **graphics & games** (every 3D transform is a matrix multiplication); **physics simulations** (force = mass x acceleration); **finance** (interest = principal x rate x time); **sensor calibration** (real_value = raw_reading x scale_factor + offset); **machine learning** (a neural network forward pass is mostly matrix multiplications). Choosing double over float matters whenever you accumulate many operations - rounding errors compound and double keeps them small.

9. Common Errors

Watch out for

- 5 / 2 = 2 instead of 2.5 - integer division because both operands are int.
- Truncation when assigning a double result to an int variable.
- Mixing float and double - prefer double unless RAM is constrained.
- Wrong result for very large numbers - overflow beyond double range.
- Catastrophic cancellation for tiny numbers - use a numerically stable algorithm.

10. Practice Question

Try this

Read three doubles representing the **length, width and height of a box** and print its **volume** and **surface area**. Hint: volume = l * w * h; surface = 2 * (l*w + w*h + h*l).

Program 06 - Swap Two Numbers Using a Third Variable

Program Number	6
Title	Swap Two Numbers Using a Third Variable
Difficulty	Beginner
File	06_*.cpp

1. Aim

To swap the values of two variables using a temporary third variable, introducing the assignment operator = and the classic three-step swap idiom.

2. Concepts Used

1. Variable assignment with =
2. Temporary / helper variable
3. Three-step swap idiom
4. Printing values before and after to verify the swap

3. Problem Statement

Read two integers from the user, display them, swap their values using a third variable, and display them again.

4. Algorithm

1. Read a and b.
2. Print "Before swap: a = ..., b = ...".
3. temp = a; // save a's value
4. a = b; // overwrite a with b
5. b = temp; // restore original a into b
6. Print "After swap: a = ..., b = ...".

5. Source Code

```
#include <iostream> using namespace std; int main() { int a = 0, b = 0, temp = 0; cout << "Enter two integers: "; cin >> a >> b; cout << "Before swap: a = " << a << ", b = " << b << endl; temp = a; a = b; b = temp; cout << "After swap: a = " << a << ", b = " << b << endl; return 0; }
```

6. Line-by-Line Explanation

int a = 0, b = 0, temp = 0;

Three integer variables are declared and zero-initialized. The temp variable is the **helper** that will hold a's value temporarily while we overwrite a.

cout << "Before swap: ..." << endl;

We print the values *before* the swap. This is critical for verifying that the swap actually worked - without "before" and "after" prints, you cannot tell if anything changed.

temp = a;

This is **step 1 of the swap**. We copy a's value into temp so that a's original value is preserved somewhere safe. After this line, temp == a (originally) and a is unchanged.

a = b;

Step 2. We overwrite a with b's value. After this line, a == b (the original b). But a's original value is *still safely stored in temp*.

b = temp;

Step 3. We copy the saved original a (from temp) into b. After this line, b == original a. The swap is now complete: a holds b's original value and b holds a's original value.

Why this ordering matters

If you did a = b; *before* temp = a;, you would lose a's original value forever - both a and b would end up equal to b. The order temp = a; a = b; b = temp; is the **only correct sequence** with three assignments.

7. Sample Input / Output

Input	5 9
Output	`Enter two integers: Before swap: a = 5, b = 9 After swap: a = 9, b = 5`

8. Real-World Application

Where this is used

Swapping is a fundamental building block of countless algorithms: **sorting algorithms** (bubble sort, insertion sort, quicksort partitioning); **graphics double-buffering** (front and back frame buffers swapped each frame); **concurrency** (swap primitives underlie lock-free data structures); **undo/redo** (current state and backup state are swapped on each undo step); **cryptography** (some ciphers like RC4 swap bytes in a key schedule). C++ provides std::swap(a, b); which does this for you.

9. Common Errors

Watch out for

- Did a = b; before saving a - both variables end up equal to b.
- Wrote a, b = b, a - that is Python syntax, not C++.
- Forgot the "before" print - looks like nothing changed.
- temp not initialized - harmless here but bad style.

10. Practice Question

Try this

Swap two integers **without** using a third variable, using (a) the arithmetic method a = a + b; b = a - b; a = a - b; (watch for overflow) and (b) the XOR method a ^= b; b ^= a; a ^= b; (overflow-safe). Verify both produce the same result as the temp-variable approach for inputs like 7 13 and -5 9.

Program 07 - Area and Perimeter of a Rectangle

Program Number	7
Title	Area and Perimeter of a Rectangle
Difficulty	Beginner
File	07_*.cpp

1. Aim

To read the length and breadth of a rectangle from the user and compute both its area (length x breadth) and perimeter (2 x (length + breadth)).

2. Concepts Used

1. Multiple variables of the same type declared together
2. Arithmetic operators * and + and parentheses
3. Two independent computations from the same input
4. Formatted multi-line output

3. Problem Statement

Write a C++ program that reads the length and breadth of a rectangle and prints its area and perimeter.

4. Algorithm

1. Declare double variables length, breadth, area, perimeter.
2. Prompt and read length and breadth.
3. Compute area = length * breadth.
4. Compute perimeter = 2 * (length + breadth).
5. Print both results with labels.

5. Source Code

```
#include <iostream> using namespace std; int main() { double length = 0.0, breadth = 0.0; double area = 0.0, perimeter = 0.0; cout << "Enter length and breadth of rectangle: "; cin >> length >> breadth; area = length * breadth; perimeter = 2 * (length + breadth); cout << "Area = " << area << endl; cout << "Perimeter = " << perimeter << endl; return 0; }
```

6. Line-by-Line Explanation

double length = 0.0, breadth = 0.0;

The two dimensions of the rectangle are stored as double because real-world lengths are often not integers (e.g. 5.5 metres). Both are declared in one statement, separated by commas.

double area = 0.0, perimeter = 0.0;

Two more double variables hold the results. Even though we will assign them before reading, initializing at declaration is good hygiene.

cin >> length >> breadth;

The user enters two numbers separated by space or Enter. cin reads them into length and breadth respectively.

area = length * breadth;

The geometric formula for the area of a rectangle. The * operator multiplies the two double values; the result is double.

perimeter = 2 * (length + breadth);

The perimeter formula is 2 x (l + w). **Parentheses are essential here** because * has higher precedence than +. Without parentheses, 2 * length + breadth would compute (2 * length) + breadth - mathematically wrong.

Two cout statements

Each result is printed on its own line with a descriptive label. Aligning the = sign with spaces makes the output easier to read.

7. Sample Input / Output

Input	5 3
Output	`Enter length and breadth of rectangle: Area = 15 Perimeter = 16`

8. Real-World Application

Where this is used

Geometry calculators are everywhere: **architecture & interior design** (room area for flooring, paint quantity estimation); **agriculture** (land parcel area for fertilizer); **manufacturing** (sheet metal layout, packaging dimensions); **game development** (axis-aligned bounding boxes for collision detection); **GIS** (bounding rectangles around map features). The pattern "read inputs then compute several results then print them" is the template for thousands of utility programs.

9. Common Errors

Watch out for

- Missing parentheses in 2 * (l + w) - perimeter becomes wrong.
- Declared int area - result truncated to integer.
- Typo: cin >> length >> length; - reads the same variable twice.
- Accepts negative dimensions - validation comes in Chapter 2 with if statements.
- cout shows scientific notation for very large numbers - use fixed and setprecision.

10. Practice Question

Try this

Extend the program to also compute and print the **length of the diagonal** using Pythagoras: diagonal = sqrt(l*l + w*w). Include <cmath>. Verify: for length = 3 and breadth = 4, the diagonal should be exactly 5.

Program 08 - Celsius to Fahrenheit Converter

Program Number	8
Title	Celsius to Fahrenheit Converter
Difficulty	Beginner
File	08_*.cpp

1. Aim

To convert a temperature given in degrees Celsius to degrees Fahrenheit using the formula F = (C x 9/5) + 32, and to demonstrate the critical importance of writing 9.0 / 5.0 (not 9 / 5) to avoid integer division.

2. Concepts Used

1. Reading a double from cin
2. Arithmetic formula evaluation
3. **Integer vs floating-point division** - the #1 beginner pitfall
4. Operator precedence and parentheses

3. Problem Statement

Write a C++ program that reads a temperature in Celsius from the user and prints the equivalent temperature in Fahrenheit.

4. Algorithm

1. Declare double celsius, fahrenheit.
2. Prompt and read celsius.
3. Compute fahrenheit = (celsius * 9.0 / 5.0) + 32.0.
4. Print both values.

5. Source Code

```
#include <iostream> using namespace std; int main() { double celsius = 0.0, fahrenheit = 0.0; cout << "Enter temperature in Celsius: "; cin >> celsius; fahrenheit = (celsius * 9.0 / 5.0) + 32.0; cout << celsius << " C = " << fahrenheit << " F" << endl; return 0; }
```

6. Line-by-Line Explanation

double celsius = 0.0, fahrenheit = 0.0;

Temperatures are real numbers (37.5, -40.0, 100.0), so we use double for both variables. Initializing with 0.0 keeps the code safe even if cin fails (e.g. user types text instead of a number).

cin >> celsius;

Reads one floating-point number from the keyboard. cin skips leading whitespace and stops at the first non-numeric character.

fahrenheit = (celsius * 9.0 / 5.0) + 32.0;

The standard conversion formula F = (C x 9/5) + 32. **Look carefully at the literals: 9.0 and 5.0, not 9 and 5.**

The Integer Division Trap

If you write 9 / 5 (both int), C++ performs **integer division** and the result is 1 (the fractional part 0.8 is discarded). The formula then becomes F = C x 1 + 32 = C + 32, which is wildly wrong - for C = 37 you would get 69 F instead of the correct 98.6 F! By writing 9.0 / 5.0 (both double), C++ performs **floating-point division** and you get the correct 1.8.

Precedence

Multiplication and division have higher precedence than addition, so celsius * 9.0 / 5.0 is evaluated left-to-right as (celsius * 9.0) / 5.0 - fine. But adding explicit parentheses (celsius * 9.0 / 5.0) + 32.0 makes the formula structure obvious to the next reader.

cout << celsius << " C = " << fahrenheit << " F" << endl;

Four values are pushed into cout: the Celsius number, the unit string " C = ", the Fahrenheit number, and the unit " F". The output reads like 37 C = 98.6 F.

7. Sample Input / Output

Input	37
Output	`Enter temperature in Celsius: 37 C = 98.6 F`

8. Real-World Application

Where this is used

Temperature conversion is used in **weather apps** (APIs return Kelvin; UI converts to C or F based on locale), **thermostats and HVAC controllers**, **medical devices** (body temperature is 37 C / 98.6 F), **cooking recipes and ovens** (US ovens use F, European ones use C), and **scientific data pipelines**. The same integer-division pitfall occurs in *every* formula with a fraction: percentage calculations (mark / total * 100), averages, scaling factors.

9. Common Errors

Watch out for

- Wrote 9/5 - integer division yields 1, so F = C + 32 (wrong!).
- Declared int fahrenheit - loses the decimal.
- cin into the wrong variable.
- Wrong precedence - celsius * 9 / 5 + 32 without parens is fragile.
- Scientific notation in output for extreme values - use fixed and setprecision.

10. Practice Question

Try this

Extend the program to ask the user **which direction** to convert (C->F or F->C). Read a char for the choice and use an if statement (Chapter 2 preview). The reverse formula is C = (F - 32) x 5/9. Sanity check: -40 C and -40 F are the same temperature.

Program 09 - Size of Primitive Data Types

Program Number	9
Title	Size of Primitive Data Types
Difficulty	Beginner
File	09_*.cpp

1. Aim

To use the sizeof operator to determine the size in bytes of every primitive C++ data type, and to understand that sizes are platform dependent (the C++ standard fixes minimums, not exacts).

2. Concepts Used

1. sizeof operator (compile-time, returns size_t)
2. All primitive types: short, int, long, long long, float, double, long double, char, bool, unsigned int
3. Platform dependence of type sizes
4. Tabular output with \t

3. Problem Statement

Write a C++ program that prints a table of every primitive data type and its size in bytes on this machine.

4. Algorithm

1. Include <iostream>.
2. Print a header row with column titles.
3. For each primitive type, print sizeof(type) next to its name.
4. Use \t to align columns.

5. Source Code

```
#include <iostream> using namespace std; int main() { cout << "Data type\t\tSize (bytes)\n"; cout << "----------------------------\n"; cout << "short\t\t\t" << sizeof(short) << "\n"; cout << "int\t\t\t" << sizeof(int) << "\n"; cout << "long\t\t\t" << sizeof(long) << "\n"; cout << "long long\t\t" << sizeof(long long) << "\n"; cout << "float\t\t\t" << sizeof(float) << "\n"; cout << "double\t\t\t" << sizeof(double) << "\n"; cout << "long double\t\t" << sizeof(long double) << "\n"; cout << "char\t\t\t" << sizeof(char) << "\n"; cout << "bool\t\t\t" << sizeof(bool) << "\n"; cout << "unsigned int\t\t" << sizeof(unsigned int) << "\n"; return 0; }
```

6. Line-by-Line Explanation

sizeof(short)

sizeof is a **compile-time operator** - it does not run any code at runtime. The compiler replaces sizeof(short) with the literal number 2 (on essentially all modern platforms) before the program is even linked. The result has type size_t, an unsigned integer type wide enough to describe the size of any object.

Why sizes vary by platform

The C++ standard fixes only **minimum sizes** and the relative ordering (1 == sizeof(char) <= sizeof(short) <= sizeof(int) <= sizeof(long) <= sizeof(long long)). It does *not* fix exact sizes. Windows uses the LLP64 data model (long stays 4 bytes even on 64-bit), while Linux/macOS use LP64 (long is 8 bytes on 64-bit). This is a frequent portability trap.

sizeof(char) is always 1

By definition, sizeof(char) == 1 on every C++ implementation. This is the fundamental unit of memory in C++ - a "byte" in the C++ sense is exactly the storage of one char, which is at least 8 bits but may be more on exotic hardware.

Tab formatting

Each data type name is followed by enough \t characters to push the size into a second column. Because type names have different lengths (int vs unsigned int), we vary the number of tabs.

Why size_t matters

Printing sizeof(...) directly with cout works because cout knows how to print size_t. But if you printf-style it with %d, you may get a warning on 64-bit systems because size_t is 8 bytes there and int is 4 - the argument types do not match. The C++ way (cout) handles this automatically.

7. Sample Input / Output

Input	(none)
Output	`Data type Size (bytes) ---------- short 2 int 4 long 8 long long 8 float 4 double 8 long double 16 char 1 bool 1 unsigned int 4`

8. Real-World Application

Where this is used

Knowing data-type sizes is essential in **binary file formats** (PNG, WAV, ELF, MP4 - every field has a fixed byte width), **network protocols** (TCP/IP, DNS - packet headers use fixed-width fields; you must use uint16_t, uint32_t from <cstdint>), **embedded programming** (microcontrollers may have only 2 KB of RAM; choosing int8_t over int saves 75% of memory), **interoperability with C** (struct layouts depend on type sizes and padding), and **database engines** (column types INT, BIGINT, TINYINT map directly to fixed-width C++ types). Use <cstdint> types (int32_t, uint64_t) whenever a specific byte width is required.

9. Common Errors

Watch out for

- Assuming int is always 4 bytes - on 16-bit MCUs it is 2 bytes.
- Assuming long is always 8 bytes - on Windows LLP64 it is 4 bytes.
- printf("%d", sizeof(x)) warning - size_t != int on 64-bit.
- Pointer size assumed = int size - on 64-bit, pointer = 8, int = 4.
- Misaligned columns in output - use multiple \t or iomanip setw().

10. Practice Question

Try this

Add these lines and explain the output: cout << "int* (pointer)\t\t" << sizeof(int*) << "\n"; cout << "double* (pointer)\t" << sizeof(double*) << "\n"; (1) What is the size of a pointer on your machine? (2) Why is the size of an int* the same as a double*, even though int and double have different sizes? (3) Include <cstdint> and print sizeof(int32_t) and sizeof(int64_t).

Program 10 - ASCII Value of a Character

Program Number	10
Title	ASCII Value of a Character
Difficulty	Beginner
File	10_*.cpp

1. Aim

To read a single character from the user and print both the character itself and its ASCII value, demonstrating the implicit conversion between char and int in C++.

2. Concepts Used

1. char data type and character literals
2. Reading a single character with cin >> ch
3. Implicit char <-> int conversion (integer promotion)
4. The ASCII table (American Standard Code for Information Interchange)
5. Explicit cast (int)ch and (char)n

3. Problem Statement

Write a C++ program that reads a character from the user and prints the character itself and its ASCII value.

4. Algorithm

1. Declare a char variable ch.
2. Prompt and read ch with cin >> ch.
3. Print ch as a character.
4. Print (int)ch to show its ASCII code.
5. Also print the next character (char)(ch + 1) for illustration.

5. Source Code

```
#include <iostream> using namespace std; int main() { char ch = '\0'; cout << "Enter a character: "; cin >> ch; cout << "Character : " << ch << "\n"; cout << "ASCII code: " << (int)ch << "\n"; cout << "Next char : " << (char)(ch + 1) << " (ASCII " << (int)(ch + 1) << ")\n"; return 0; }
```

6. Line-by-Line Explanation

char ch = '\0';

A char variable is declared and initialized to the null character '\0' (ASCII 0). Characters use **single quotes**; the backslash \0 is an escape sequence for "null character". Initializing to \0 is safer than leaving ch uninitialized.

cin >> ch;

cin reads a single non-whitespace character into ch. Important: cin >> skips leading whitespace (spaces, tabs, newlines) by default. If you actually want to read a space character, you must use cin.get(ch) instead - that reads whatever is next in the stream, including whitespace.

cout << "Character : " << ch << "\n";

When you print a char with cout, you get the character itself - A for 'A', 5 for '5', + for '+'.

(int)ch - explicit cast to int

This is the heart of the program. Writing (int)ch tells the compiler to **interpret the byte stored in ch as an integer**. Because char and int are both integer types, the conversion is automatic (implicit) - you could even write cout << ch + 0 and get the same effect, because adding 0 triggers **integer promotion**. But the explicit cast (int)ch makes the intent obvious. For 'A', the ASCII code is 65; for 'a' it is 97; for '0' it is 48; for a space ' ' it is 32.

(char)(ch + 1) - back to char

Adding 1 to a char produces an int (because of integer promotion). Casting the result back to char with (char) lets us print it as a character again. For input 'A', ch + 1 is 66, which when printed as a char gives 'B' - the next letter in the alphabet. This trick - arithmetic on character codes - is the basis of all simple ciphers (Caesar cipher, ROT13) and case conversion.

7. Sample Input / Output

Input	A
Output	`Enter a character: Character : A ASCII code: 65 Next char : B (ASCII 66)`

8. Real-World Application

Where this is used

Character <-> code conversion is used in **cryptography** (Caesar cipher shifts each letter by N: cipher = (char)(plain + key); ROT13 is the special case key = 13), **case conversion** ('A' and 'a' differ by 32; toupper(c) = c - 'a' + 'A'), **lexical analysis in compilers** (every scanner reads characters and classifies them by ASCII range), **password hashing**, **text encoding** (UTF-8 encodes Unicode codepoints as sequences of bytes), and **serial protocols** (a microcontroller receiving bytes from UART treats each byte as either a char or an unsigned char depending on whether it represents text or raw data).

9. Common Errors

Watch out for

- Reads multiple characters into one char - only the first is stored.
- Cannot read a space - cin >> skips whitespace; use cin.get(ch).
- cout << ch + 0 prints a number, not a char - integer promotion kicked in.
- Wrong case conversion - A is 65 and a is 97; use toupper/tolower from <cctype>.
- Garbage for non-ASCII input (e.g. accented letters) - char is signed on many systems; values > 127 become negative. Use unsigned char for raw bytes.

10. Practice Question

Try this

Read a **lowercase** letter and print its **uppercase** equivalent using the ASCII relationship. Verify with the library function toupper from <cctype>: char lower, upper; cin >> lower; upper = (char)(lower - 'a' + 'A'); // manual cout << (char)toupper(lower); // library Try inputs a, z, m and confirm both methods give the same result. What happens for input 'A' (already uppercase)? Why?

Chapter Summary

Chapter 1 introduced the absolute foundations of C++ programming - the structure of a program, console input and output, primitive data types, escape sequences, comments and the char-to-int conversion. These ten programs may look small in isolation, but together they cover every building block that the rest of the course depends on.

You should now be comfortable with the boilerplate of every C++ program: **#include <iostream>**, **using namespace std;**, **int main() { ... }**, and **return 0;**. You can print text with cout and the << operator, read values from the keyboard with cin and the >> operator, and you understand that every statement must end with a semicolon.

Three lessons deserve special emphasis because they cause the majority of beginner bugs. First, **initialize variables at declaration** - reading from an uninitialized variable is undefined behaviour. Second, **use 9.0 / 5.0, never 9 / 5**, in any formula involving a fraction - integer division silently discards the decimal part and produces wrong results. Third, **use single quotes for chars and double quotes for strings** - the two are different types in C++.

You have also learned that C++ is a case-sensitive language, that type sizes are platform-dependent (so portable code uses fixed-width types from <cstdint> when a specific byte width is required), and that characters are stored as small integers - the same trick that lets you print the ASCII value of 'A' as 65 also lets you build simple ciphers and case converters.

Practice Exercises - Round-Up

The following table collects the practice question from each program in this chapter. Tackle them in order; each one builds on the program it extends.

#	Practice Exercise
01	Modify the program to print your full name and your college name on two separate lines using a single cout statement. Hint: chain endl between two strings: cout << "My Name" << endl << "My College" << endl;
02	Write a program that prints the ASCII art of a triangle using escape sequences and a single cout statement: /\ / \ /____\ Hint: you need to escape each backslash. Use leading spaces for alignment and between lines.
03	Add a variable of type unsigned int and one of type long long. Print their sizes and try to store a value larger than 2 billion in each (e.g. 3000000000). Observe what happens if you store a value larger than INT_MAX into a plain int.
04	Extend the program to read three integers and print both their sum and average (the average should be a double). Hint: use sum / 3.0, not sum / 3, to avoid integer division!
05	Read three doubles representing the length, width and height of a box and print its volume and surface area. Hint: volume = l * w * h; surface = 2 * (l*w + w*h + h*l).
06	Swap two integers without using a third variable, using (a) the arithmetic method a = a + b; b = a - b; a = a - b; (watch for overflow) and (b) the XOR method a ^= b; b ^= a; a ^= b; (overflow-safe). Verify both produce the same result as the temp-variable approach for inputs like 7 13 and -5 9.
07	Extend the program to also compute and print the length of the diagonal using Pythagoras: diagonal = sqrt(l*l + w*w). Include <cmath>. Verify: for length = 3 and breadth = 4, the diagonal should be exactly 5.
08	Extend the program to ask the user which direction to convert (C->F or F->C). Read a char for the choice and use an if statement (Chapter 2 preview). The reverse formula is C = (F - 32) x 5/9. Sanity check: -40 C and -40 F are the same temperature.
09	Add these lines and explain the output: cout << "int* (pointer) " << sizeof(int*) << " "; cout << "double* (pointer) " << sizeof(double*) << " "; (1) What is the size of a pointer on your machine? (2) Why is the size of an int* the same as a double*, even though int and double have different sizes? (3) Include <cstdint> and print sizeof(int32_t) and sizeof(int64_t).
10	Read a lowercase letter and print its uppercase equivalent using the ASCII relationship. Verify with the library function toupper from <cctype>: char lower, upper; cin >> lower; upper = (char)(lower - 'a' + 'A'); // manual cout << (char)toupper(lower); // library Try inputs a, z, m and confirm both methods give the same result. What happens for input 'A' (already uppercase)? Why?

Conclusion and Next Steps

Congratulations on completing Chapter 1. You now have a working C++17 toolchain (GCC, VS Code), a project repository structure on GitHub, and ten small but real programs that demonstrate the core mechanics of input, output, variables and arithmetic. You are no longer just reading about C++ - you are writing it.

Before moving on, make sure you can do all of the following without consulting the notes: write a Hello World program from scratch; read two numbers from the keyboard and print their sum, product, difference and quotient; swap two variables using a third variable; declare variables of every primitive type and print their sizes; read a character and print its ASCII value. If any of these feels uncertain, re-read the relevant program and complete its practice question first.

In **Chapter 2 - Operators and Conditional Statements** you will add decision-making to your programs. You will learn the full set of arithmetic, relational, logical, bitwise and assignment operators, the if / else if / else ladder, the switch statement and the ternary operator. With conditionals, your programs stop being straight-line scripts and start behaving like real software - validating input, choosing between alternatives, and responding differently to different situations.

Take a moment to commit your Chapter 1 work to Git and push it to GitHub (the commands are listed in the project README). A clean, well-documented commit history is itself a portfolio piece - future employers and collaborators will judge your professionalism by how neatly your repository is organised. Then return here and say *"Ready for Chapter 2"* to continue the course.

Next chapter: **Operators and Conditional Statements** - 10 programs on arithmetic, relational, logical and bitwise operators, if/else, switch and the ternary operator.